

PROJET : Simulation de MOTOROLA 6809

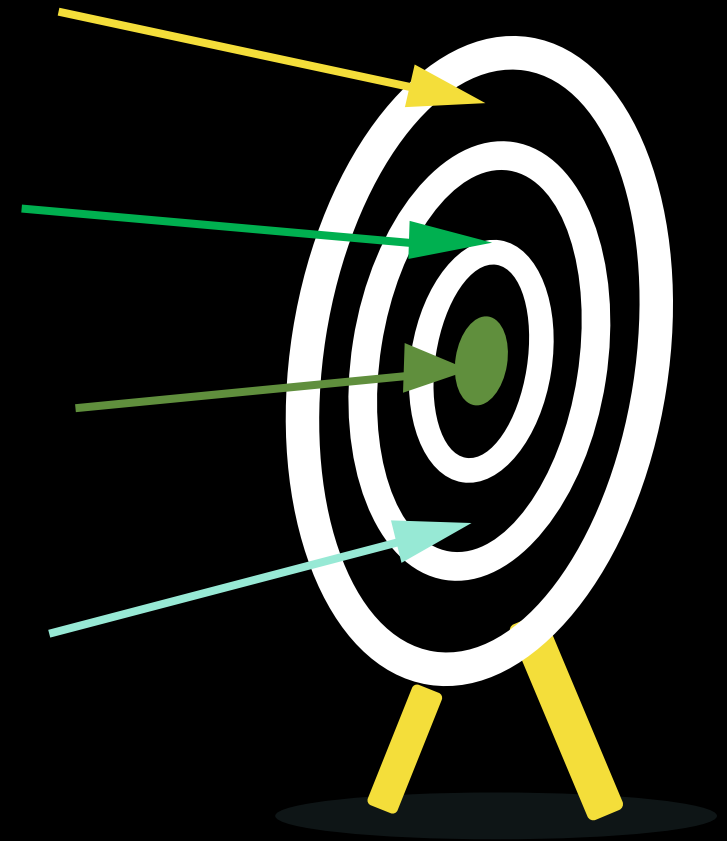


présenté par :
KAWTAR IMZILNE
FATNA ELMAZZOUZI
HAJAR ID-IDAR

Encadré par :
Mr. BENALLA Hicham

LE PLAN

- 1.Introduction
- 2.Interface graphique
- 3.Cœur de la simulation
- 4.Modes d'adressage
- 5.Algorithme de compilation
- 6.Conclusion



INTRODUCTION



L'objectif de ce projet est de créer un simulateur pour le processeur Motorola 6809 en se servant du langage Java. L'objectif principal est d'acquérir une compréhension approfondie du fonctionnement interne d'un microprocesseur en adoptant une méthode pratique et visuelle. La simulation offre la possibilité d'observer l'exécution des commandes, la gestion de la mémoire et le fonctionnement des divers éléments

L'INTERFACE GRAPHIQUE

Interface
principale

Interface de
mémoire ROM

Interface de
mémoire ROM

Interface principale

Assembleur Affichage

PC : 0000

S : 0000 U : 0000

A : 00 D : 0000 B : 00

DP: 00 00000100

X: 0000 Y : 0000

UAL

EFHI NZVC

ROM

Adresse	Valeur
FC00	FF
FC01	FF
FC02	FF
FC03	FF
FC04	FF
FC05	FF
FC06	FF
FC07	FF
FC08	FF
FC09	FF
FC0A	FF
FC0B	FF
FC0C	FF

RAM

Adresse	Valeur
0000	00
0001	00
0002	00
0003	00
0004	00
0005	00
0006	00
0007	00
0008	00
0009	00
000A	00
000B	00
000C	00

Assembleur

Reset

Pas_à_Pas executer

Interface
D'architecture
interne

Interface
d'Assembleur

CŒUR DE LA SIMULATION

La Classe Assembleur

Contient toutes les instructions du microprocesseur. Traduit le code assembleur en opérations exécutables. Gère les modes d'adressage et la conversion entre hexadécimal, décimal et binaire. Vérifie les flags et l'emplacement des registres dans la RAM et ROM.

La Classe CPU

Contient tous les registres du microprocesseur (A, B, X, Y...). Permet de lire et modifier les registres (getRegistre() et setRegistre()). Simule le traitement interne des instructions.

La Classe RAM

Stocke temporairement les données utilisées par le CPU. Permet la lecture et l'écriture des valeurs pendant l'exécution du programme. Représente la mémoire volatile du microprocesseur.

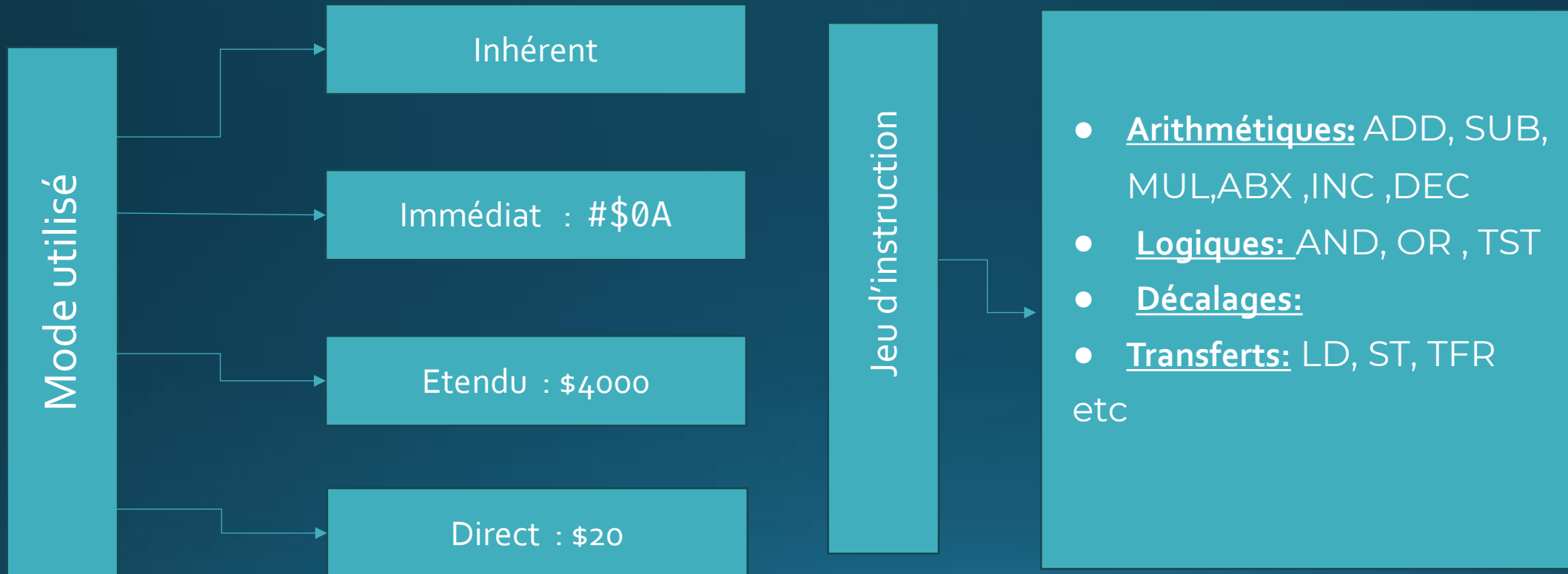
La Classe ROM

Stocke les instructions et leurs adresses (lecture seule). Contient le programme à exécuter. Permet au CPU de lire les instructions pour les traiter.

La Classe MAIN

Permet à l'utilisateur de contrôler le programme. Boutons principaux : Exécuter, Exécuter pas à pas, Reset. Affiche les registres, la RAM et la ROM pour suivre l'exécution.

LED MODE D'ADRESSAGE



ÉDITEUR ASSEMBLEUR

CLR \$14 ; On met \$14 à 0
INC \$14 ; On incrémente \$14 → \$14 = 1
LDA #\$80 ; On charge A avec 0x80
STA \$15 ; On stocke A dans \$15 ; \$15 = 0x80
TST \$15 ; On teste \$15, valeur inchangée
LDX #\$0040 ; On charge X avec 0x0040
STX \$18 ; On stocke X dans \$18 ; \$18 = 0x0040
NOP ; Rien ne change
LDA #\$04 ; On charge A avec 0x04
LDB \$03 ; On charge B avec la valeur de \$03
MUL ; On multiplie A*B ; résultat en D
NOP ; Rien ne change
END ; Fin du programme

- ☐ Lecture du code assembleur
- ☐ Analyse et vérification
- ☐ Détection des erreurs
- ☐ Traduction en code machine (opcodes)
- ☐ Calcul et organisation en mémoire
- ☐ Chargement du programme en ROM

DECODAGE DES INSTRUCTIONS



- ❖ Le système de décodage basé sur l'opcode détecte automatiquement le **mode d'adressage** (immédiat, direct, étendu, indexé) et gère les **jeux d'instructions**

ROM	
FC00	86
FC01	80

Une instruction textuelle comme **LDA #\$80** est convertie en valeurs **hexadécimales**.

Opcode : La valeur 86 est générée pour représenter l'ordre "Charger l'accumulateur".

Stockage : Ces données sont injectées à des adresses précises (ex: FC00) dans la mémoire ROM pour être prêtes à l'emploi.

Le CPU l'exécute dans une **boucle Fetch-Decode-Execute**, mettant à jour registres et RAM,, jusqu'à l'arrêt. END



DECODAGE DES INSTRUCTION



- ❖ la RAM conserve les valeurs utilisées pendant l'exécution du programme (variables, registres intermédiaires, résultats de calcul).

RAM	
0000	00
0001	34

Une instruction textuelle comme STA #\$0001 est convertie en **valeurs hexadécimales**.

Opcode : La valeur 97 est générée pour représenter l'ordre "Stocker l'accumulateur".

Stockage ROM : Ces données sont placées à une adresse précise (ex: FCo1) dans la mémoire ROM, prêtes à l'emploi.

RAM : Lorsque l'instruction est exécutée, la valeur de l'accumulateur est **écrite dans la RAM** à l'adresse \$0001, où elle peut être lue ou modifiée par d'autres instructions.



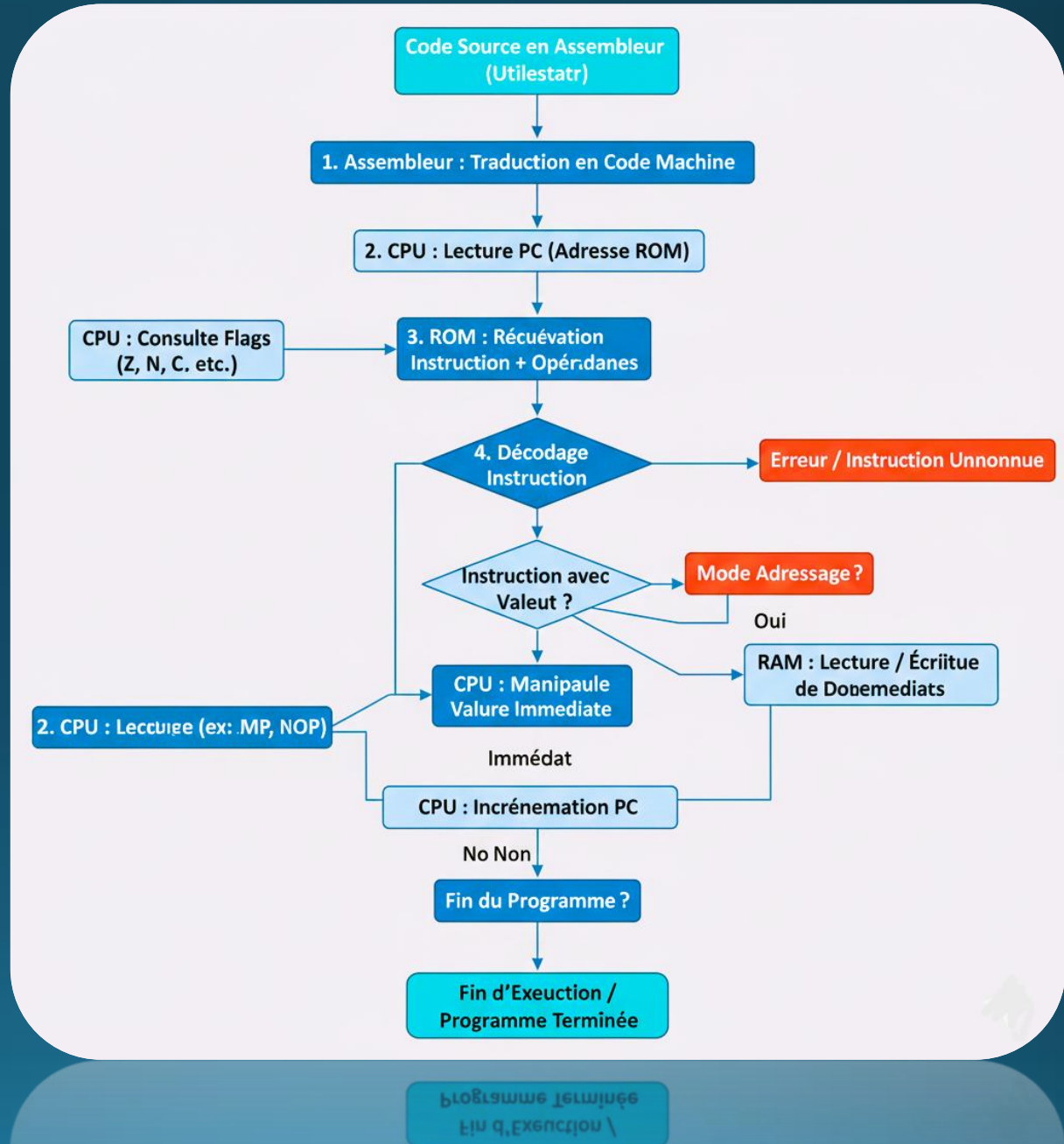
LES CONTRAINTE ET RÉPONSE TECHNIQUE



CONTRAINTE	✓ RÉPONSE TECHNIQUE
Interaction utilisateur via GUI	Boutons dans la classe Interface
Gestion des instructions	Classe Assembleur avec switch-case
Modes d'adressage variés	Méthodes dédiées dans Assembleur
Gestion des registres	Classe Registres avec getters/setters
Lecture/écriture mémoire	Classes RAM et ROM avec méthodes obtenir () et modifier ()
Gestion des erreurs	Vérification dans Assembleur + messages GUI



ALGORITHME DE COMPILATION



PROGRAMMES DE TEST

Assembleur

Affichage

ARCH...

PC : FC17

END

S : 2000 U : 0000

A : 03

D : 0000

B : 20

DP: 00 00000000

X: 1234 Y : 0000

EFHINZVC

UAL

ROM

Adresse	Valeur
FC00	86
FC01	25
FC02	C6
FC03	10
FC04	8E
FC05	12
FC06	34
FC07	10
FC08	CE
FC09	20
FC0A	00
FC0B	B7
FC0C	00

RAM

Adresse	Valeur
0000	00
0001	00
0002	00
0003	25
0004	00
0005	00
0006	00
0007	00
0008	00
0009	00
000A	00
000B	00
000C	00

Assembleur

Reset

Pas_à_Pas

executer

LDA #\$25
LDB #\$10
LDX #\$1234
LDS #\$2000
STA \$0003
LDB #\$80
SUBB #\$30
LDA #\$0A
INCA
DECA
MUL
END



TEST ET VALIDATION



Mode ✓
d'adressage
fonctionne

✓ Gestion
des erreurs

✓ GUI
(interface)

✓ Les
instruction
fonctionne

RESULTAT :

- Test des instructions (arithmétiques, logiques, transferts) correct
- Vérification des modes d'adressage
- Contrôle des flags après chaque opération



CONCLUSION

En conclusion, ce projet a permis de réaliser une simulation fonctionnelle du processeur Motorola 6809. Il a contribué à améliorer la compréhension de l'architecture des processeurs et de la programmation orientée objet. Des améliorations futures pourraient inclure l'ajout de nouvelles instructions et une exécution pas à pas plus avancée.